

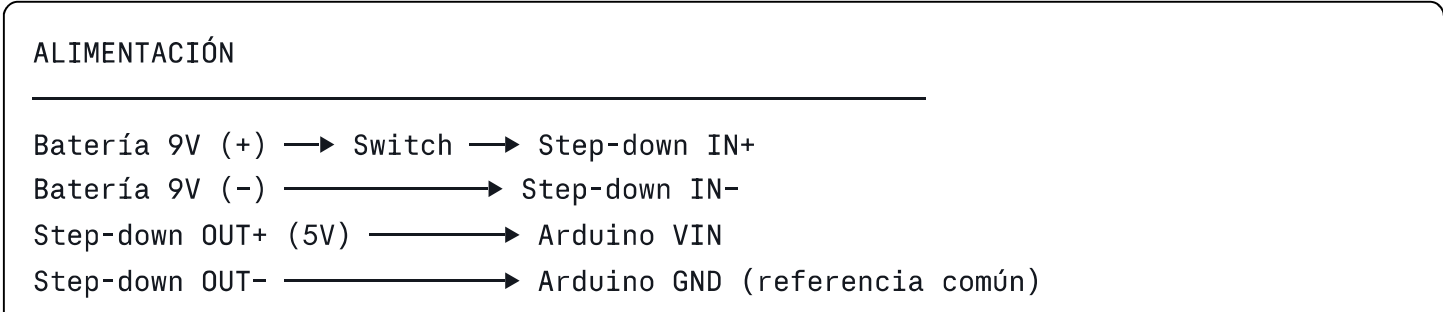
# Detector de Protocolo RF — Versión 4 Final

315MHz + 433MHz | Display OLED | Batería con indicador | Botón reset

## Lista de materiales

| Cant  | Componente                   | Detalle                         | USD aprox    |
|-------|------------------------------|---------------------------------|--------------|
| 1     | Arduino Nano                 | Con cable USB                   | \$3-5        |
| 1     | Módulo receptor RF 433.92MHz | RXB6 o XY-MK-5V                 | \$1-2        |
| 1     | Módulo receptor RF 315MHz    | XY-MK-5V 315MHz                 | \$1-2        |
| 1     | Pantalla OLED 0.96"          | SSD1306, I2C, 128×64px          | \$2-3        |
| 1     | Batería 9V Li-ion recargable | Con LEDs de carga integrados    | \$4-6        |
| 1     | Módulo step-down DC-DC       | Mini ajustable, 9V → 5V         | \$0,50-1     |
| 1     | Switch ON/OFF                | Mini deslizante o de palanca    | \$0,30       |
| 1     | Pulsador NA                  | Normalmente abierto, para reset | \$0,20       |
| 1     | Resistencia 10kΩ             | Divisor voltaje batería R1      | \$0,02       |
| 1     | Resistencia 4.7kΩ            | Divisor voltaje batería R2      | \$0,02       |
| 1     | Caja plástica                | 100×60×25mm aprox               | \$2-3        |
| 1     | PCB perforada                | Para soldar                     | \$0,50-1     |
| —     | Cables varios colores        | —                               | \$0,50       |
| Total |                              |                                 | ~\$15-23 USD |

## Diagrama de conexiones



#### RECEPTOR 433.92MHz

---

VCC → Arduino 5V  
GND → Arduino GND  
DATA → Arduino D2

#### RECEPTOR 315MHz

---

VCC → Arduino 5V  
GND → Arduino GND  
DATA → Arduino D3

#### PANTALLA OLED SSD1306 (I2C)

---

VCC → Arduino 3.3V  
GND → Arduino GND  
SDA → Arduino A4  
SCL → Arduino A5

#### MEDICIÓN DE BATERÍA

---

Bat 9V (+) → R1 (10kΩ) —┐→ R2 (4.7kΩ) → GND  
                                 └→ Arduino A0

#### BOTÓN RESET

---

Un terminal → Arduino D6  
Otro terminal → GND  
(sin resistencia externa, usa el pull-up interno del Arduino)

---

## Código completo

cpp

```
#include <RCSwitch.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

RCSwitch sw433 = RCSwitch();
RCSwitch sw315 = RCSwitch();

const int PIN_RESET = 6;
const int PIN_VBAT = A0;

unsigned long codigos[3] = {0, 0, 0};
int indice = 0;
String freqDetectada = "";
bool esperandoReset = false;

int medirBateria() {
    int raw = analogRead(PIN_VBAT);
    float vMedido = raw * (5.0 / 1023.0);
    float vBat = vMedido * (10.0 + 4.7) / 4.7;
    int pct = (int)((vBat - 6.5) / (8.4 - 6.5) * 100.0);
    return constrain(pct, 0, 100);
}

void dibujarBateria(int pct) {
    display.drawRect(106, 0, 18, 8, SSD1306_WHITE);
    display.fillRect(124, 2, 2, 4, SSD1306_WHITE);
    int ancho = (int)(14.0 * pct / 100.0);
    if (ancho > 0) display.fillRect(108, 2, ancho, 4, SSD1306_WHITE);
}

void cabecera() {
    int bat = medirBateria();
    dibujarBateria(bat);
    display.setCursor(0, 0);
    display.print("BAT:");
    display.print(bat);
    display.print("%");
    display.drawLine(0, 10, 128, 10, SSD1306_WHITE);
}
```

```
void mostrarEspera() {
    display.clearDisplay();
    display.setTextSize(1);
    display.setTextColor(SSD1306_WHITE);
    cabecera();
    display.setCursor(0, 14);
    display.println("Apriete el control");
    display.setCursor(0, 26);
    display.println("del porton 3 veces");
    display.setCursor(0, 42);
    display.print("Detectados: ");
    display.print(min(indice, 3));
    display.print("/3");
    display.setCursor(0, 54);
    display.print("315 y 433 MHz activo");
    display.display();
}

void mostrarCompatible(unsigned long codigo, int protocolo, String freq) {
    display.clearDisplay();
    display.setTextSize(1);
    display.setTextColor(SSD1306_WHITE);
    cabecera();
    display.setTextSize(2);
    display.setCursor(0, 13);
    display.println("COMPATIBLE");
    display.setTextSize(1);
    display.drawLine(0, 33, 128, 33, SSD1306_WHITE);
    display.setCursor(0, 37);
    display.print("Freq: ");
    display.print(freq);
    display.setCursor(0, 48);
    display.print("Proto: ");
    display.print(protocolo);
    display.setCursor(0, 57);
    display.print("Puls. RESET p/nuevo");
    display.display();
}

void mostrarIncompatible(int protocolo, String freq) {
    display.clearDisplay();
    display.setTextSize(1);
    display.setTextColor(SSD1306_WHITE);
    cabecera();
    display.setCursor(0, 13);
    display.println("NO COMPATIBLE");
    display.drawLine(0, 23, 128, 23, SSD1306_WHITE);
```

```

display.setCursor(0, 27);
display.println("Rolling Code detectado");
display.setCursor(0, 39);
display.print("Freq: ");
display.print(freq);
display.setCursor(0, 50);
display.println("RS400 no funciona.");
display.setCursor(0, 58);
display.print("Puls. RESET p/nuevo");
display.display();
}

void resetearDetector() {
    indice = 0;
    codigos[0] = codigos[1] = codigos[2] = 0;
    freqDetectada = "";
    esperandoReset = false;
    sw433.resetAvailable();
    sw315.resetAvailable();
    mostrarEspera();
}

void procesarResultado(unsigned long codigo, int protocolo) {
    bool esFijo = (codigos[0] == codigos[1]) && (codigos[1] == codigos[2]);
    esperandoReset = true;

    if (esFijo) {
        Serial.println(">>> CODIGO FIJO - COMPATIBLE");
        mostrarCompatible(codigo, protocolo, freqDetectada);
    } else {
        Serial.println(">>> ROLLING CODE - NO COMPATIBLE");
        mostrarIncompatible(protocolo, freqDetectada);
    }
}

void setup() {
    Serial.begin(9600);
    pinMode(PIN_RESET, INPUT_PULLUP);

    sw433.enableReceive(0); // D2
    sw315.enableReceive(1); // D3

    if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
        Serial.println("Error OLED");
        while (true);
    }
}

```

```

display.clearDisplay();
display.setTextSize(1);
display.setTextColor(SSD1306_WHITE);
display.setCursor(10, 8);
display.println("DETECTOR RF 433/315");
display.setCursor(20, 24);
display.println("v4.0 - Iniciando");
display.display();
delay(1500);

mostrarEspera();
}

void loop() {

    // Botón reset — activo en LOW por pull-up interno
    if (digitalRead(PIN_RESET) == LOW) {
        delay(50); // anti-rebote
        if (digitalRead(PIN_RESET) == LOW) {
            resetearDetector();
            while (digitalRead(PIN_RESET) == LOW); // esperar que suelten
        }
    }

    // Si estamos mostrando resultado, ignorar señales hasta reset
    if (esperandoReset) return;

    // Chequear 433MHz
    if (sw433.available()) {
        unsigned long cod = sw433.getReceivedValue();
        int proto = sw433.getReceivedProtocol();
        sw433.resetAvailable();
        if (cod == 0) return;
        if (freqDetectada == "") freqDetectada = "433.92 MHz";
        codigos[indice % 3] = cod;
        indice++;
        Serial.print("433MHz | Cod:"); Serial.print(cod);
        Serial.print(" Proto:"); Serial.println(proto);
        mostrarEspera();
        if (indice >= 3) procesarResultado(cod, proto);
    }

    // Chequear 315MHz
    if (sw315.available()) {
        unsigned long cod = sw315.getReceivedValue();
        int proto = sw315.getReceivedProtocol();
        sw315.resetAvailable();
    }
}

```

```
if (cod == 0) return;
if (freqDetectada == "") freqDetectada = "315 MHz";
codigos[indice % 3] = cod;
indice++;
Serial.print("315MHz | Cod:"); Serial.print(cod);
Serial.print(" Proto:"); Serial.println(proto);
mostrarEspera();
if (indice >= 3) procesarResultado(cod, proto);
}
}
```

---

## Librerías a instalar

En Sketch → Include Library → Manage Libraries:

- `rc-switch` (sui77)
- `Adafruit SSD1306` (Adafruit) → al instalar, clic en **Install All**
- `Adafruit GFX Library` (Adafruit)

---

## Botón reset — detalles

- Pulsador normalmente abierto (NA), el más común
- Un terminal a **D6**, el otro a **GND**
- Sin resistencia externa: el código activa el pull-up interno del Arduino
- El resultado queda en pantalla hasta que se presiona el botón
- Funciona también como reset parcial si te equivocás a mitad de las 3 pulsaciones

---

## Perforaciones en la caja

- Frente:** ventana OLED (26×14mm), pulsador reset (agujero 6mm)
- Lateral izquierdo:** switch ON/OFF
- Lateral derecho o fondo:** acceso micro-USB de la batería (para cargar sin abrir)

---

## Pantalla de espera

```
BAT: 87%      [==== ]
```

---

Apriete el control  
del porton 3 veces

Detectados: 2/3  
315 y 433 MHz activo

## Resultado compatible

BAT:87% [===== ]

COMPATIBLE

Freq: 433.92 MHz  
Proto: 1  
Puls. RESET p/nuevo

## Resultado incompatible

BAT:87% [===== ]

NO COMPATIBLE

Rolling Code detectado  
Freq: 315 MHz  
RS400 no funciona.  
Puls. RESET p/nuevo

*v4.0 Final — sin LEDs de resultado / botón reset manual / 315+433MHz / OLED / batería*